Conociendo algoritmos de Machine Learning con R

k Nearest Neighbor (kNN)

Autor: Ander Fernández

Principios sobre kNN

K Nearest Neighbors es un **algortimo de machine learning** que sirve tanto para **regresión como clasificación**.

La idea detrás del algoritmo kNN es muy sencilla: **una observación será parecida a cómo es su entorno**. Por tanto, para predecir el valor o la clase, de una observación, debemos mirar a las observaciones que están más cerca.

Principios sobre kNN

K Nearest Neighbors es un **algortimo de machine learning** que sirve tanto para **regresión como clasificación**.

La idea detrás del algoritmo kNN es muy sencilla: **una observación será parecida a cómo es su entorno**. Por tanto, para predecir el valor o la clase, de una observación, debemos mirar a las observaciones que están más cerca.

Principios sobre kNN

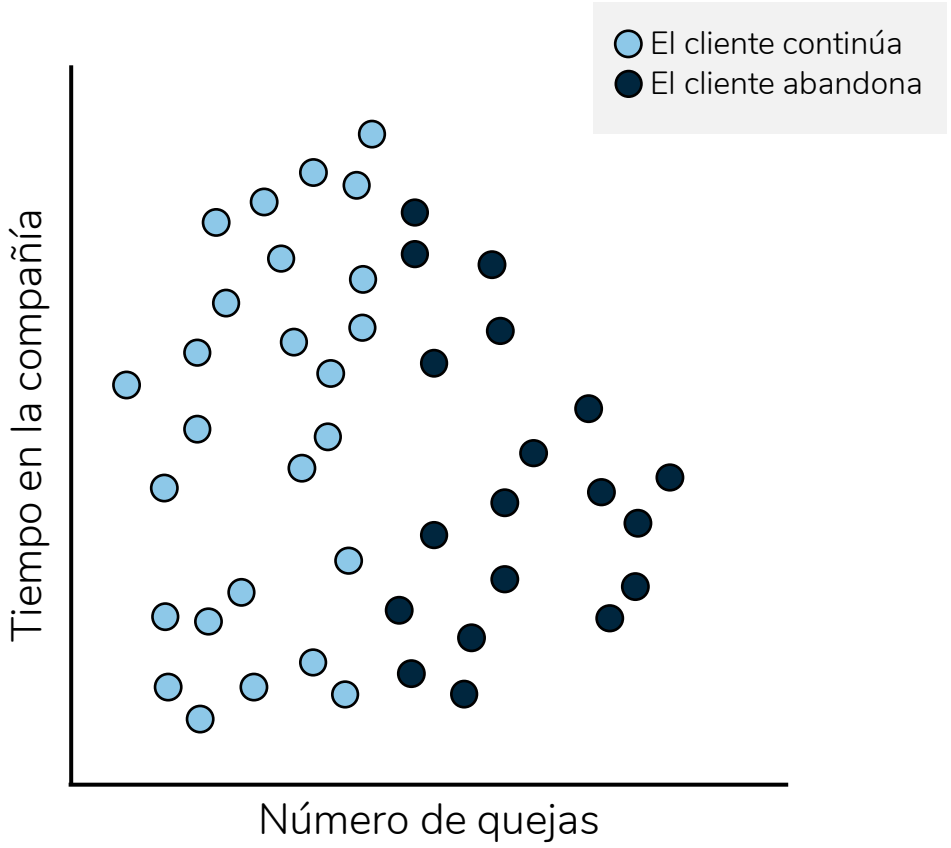
Imaginemos que tenemos una empresa y queremos predecir si un cliente dejará o no la compañía.

Para ello, hemos sacado dos datos:

- Tiempo que lleva en la compañía.
- Número de reclamaciones que ha hecho.
- Si el cliente sigue o no en la empresa.

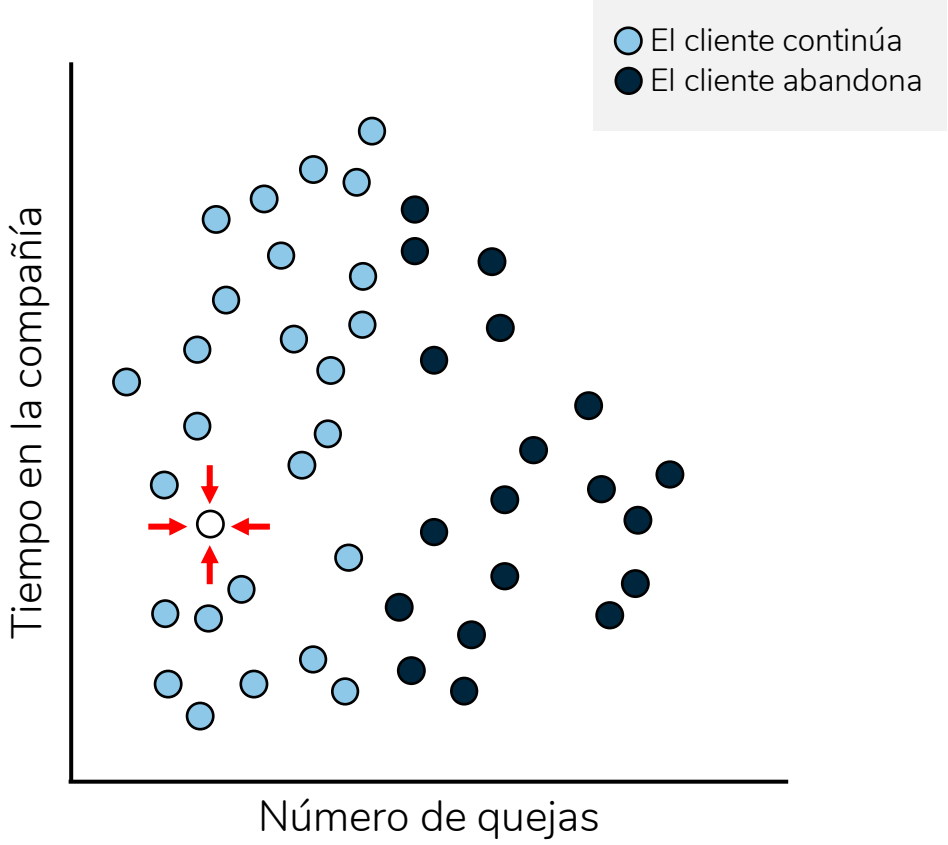
Principios sobre kNN

Si visualizamos los datos y los coloreamos según si han abandonado o no la empresa, veríamos algo así:



Principios sobre kNN

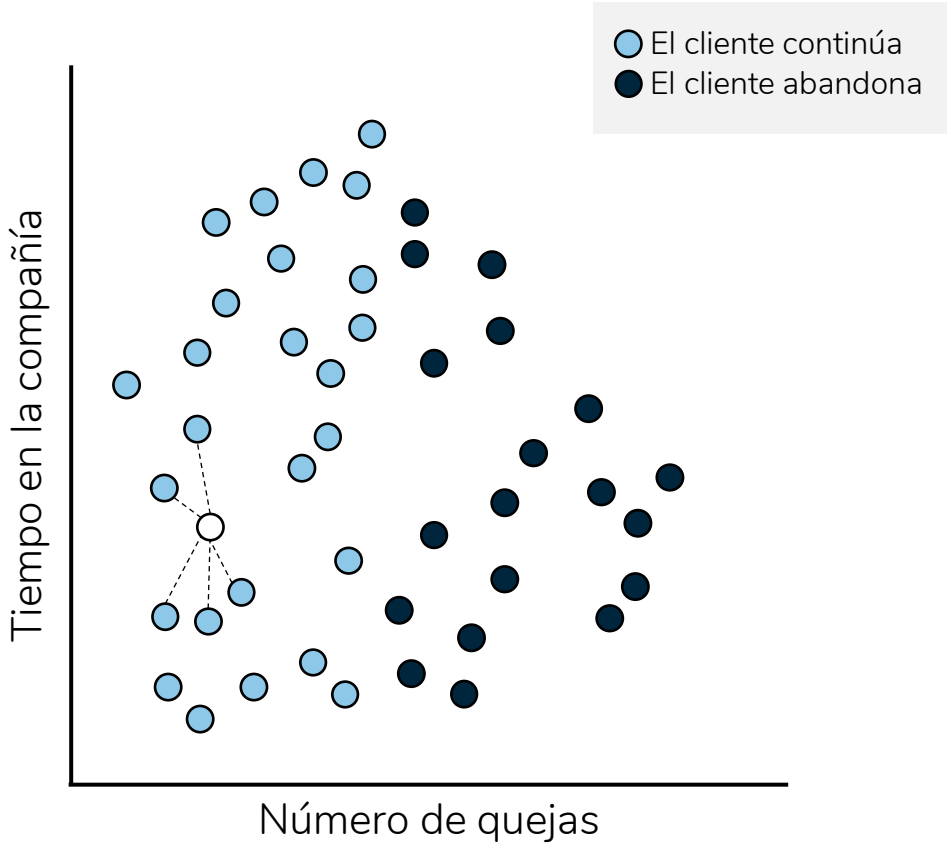
¿Qué pasaría si ahora entrase un nuevo cliente y al de un mes queremos predecir si va a abandonar la empresa o no?



Autor: Ander Fernández

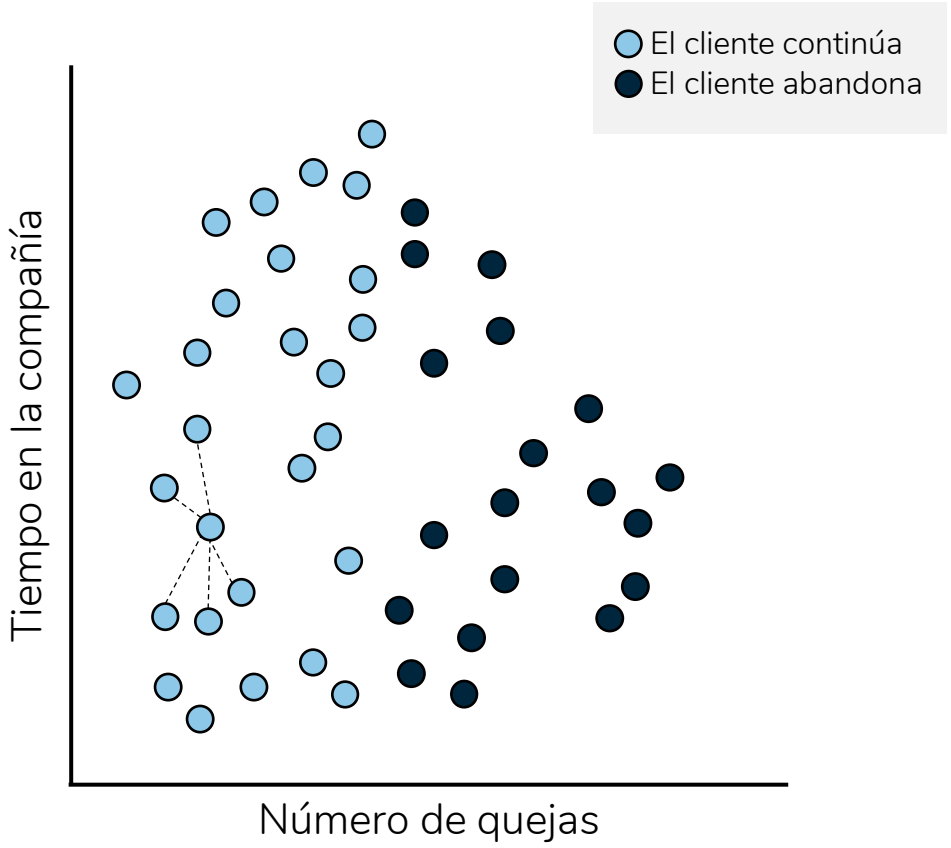
Principios sobre kNN

Podríamos fijarnos en los 5 vecinos más cercanos y predecir lo mismo que la mayoría de sus vecinos:



Principios sobre kNN

Como la gran mayoría de vecinos son clientes que continúan, nuestra predicción será que el usuario continúa en la empresa.



Autor: Ander Fernández

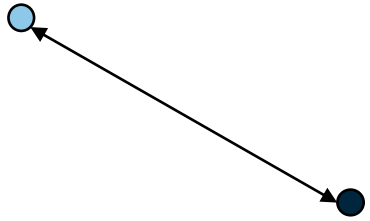
El algoritmo kNN se basa en predecir lo mismo que los vecinos más cercanos

El algoritmo kNN se basa en predecir lo mismo que los vecinos más cercanas

¿Pero cómo encontramos a esos vecinos?

Diferentes medidas de Distancia

Distancia Euclídea

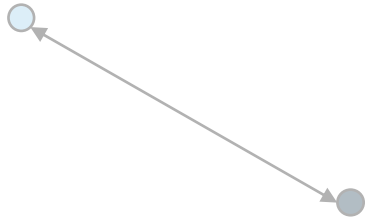


La distancia Euclídea es la distancia en línea recta más corta entre dos puntos.

Autor: Ander Fernández

Diferentes medidas de Distancia

Distancia Euclídea



La distancia Euclídea es la distancia en línea recta más corta entre dos puntos.

Distancia Manhattan

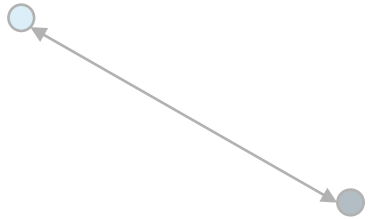
La distancia Manhattan se calcula sumando los dos catetos del triángulo.

Autor: Ander Fernández

(x_2, y_2)
(x_1, y_1)
East 73rd Street
East 72nd Street
East 71st Street
East 70th Street
East 69th Street
East 68th Street
East 67th Street
East 66th Street
East 65th Street
East 64th Street
East 63rd Street
East 62nd Street
East 61st Street
East 60th Street
East 59th Street
East 58th Street
East 57th Street
East 55th Street
East 76th Street
East 75th Street
East 74th Street
East 77th Street
East 79th Street
East Green
The Dene
Willowdell Arch
Denesmouth Arch
5th Avenue
Madison Avenue
Park Avenue
Lexington Avenue
3rd Avenue
2nd Avenue
1st Avenue
York Avenue
FDR Drive
68th Street - Hunter College (6)
Lexington Avenue-63rd Street (F, Q)
59th Street-Lexington Avenue (N,R,W)
Roosevelt Island Tramway
NY 25
Gristedes
Food Emporium
Ethan Allen
Weill Cornell Medical College
NewYork-Presbyterian Hospital
Rockefeller University
East River
West Road
Roosevelt Island Greenway
John Finley Walk

Diferentes medidas de Distancia

Distancia Euclídea

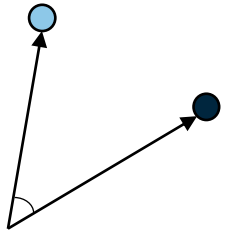


La distancia Euclídea es la distancia en línea recta más corta entre dos puntos.

Distancia Manhattan

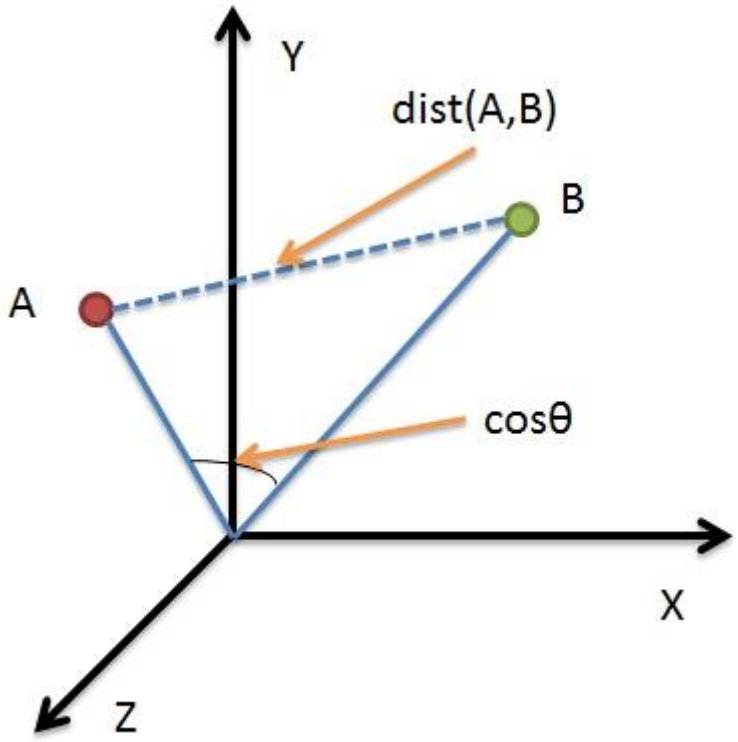
La distancia Manhattan se calcula sumando los dos catetos del triángulo.

Distancia Coseno



La distancia coseno se basa en calcular el ángulo entre las dos observaciones.

Autor: Ander Fernández


Y
dist(A,B)
B
A
cosθ
X
Z

Ya entendemos cómo funciona este algoritmo y las distancias que utiliza, así que....

¡Programemos nuestro algoritmo kNN!

Programar el algoritmo kNN

1. Programar las medidas de distancia que queramos usar
2. Encontrar los k vecinos más cercanos
3. Hacer la predicción
4. Juntar todo

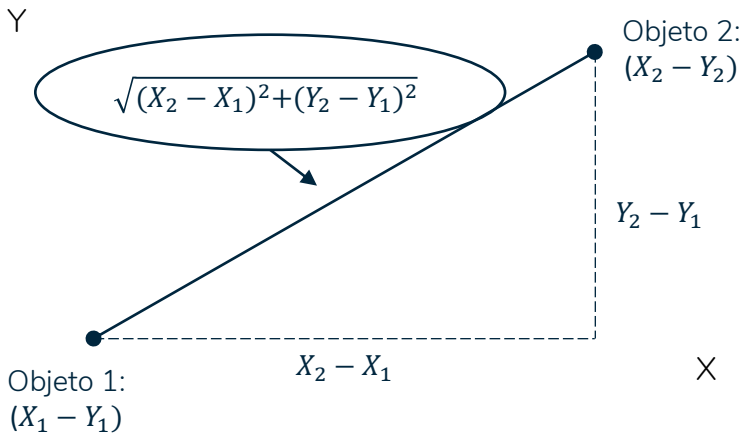
Programar el algoritmo kNN

1. Programar las medidas de distancia que queramos usar
2. Encontrar los k vecinos más cercanos
3. Hacer la predicción
4. Juntar todo

Programar el algoritmo kNN

Distancia Euclídea

La distancia Euclídea se calcula siguiendo el teorema de Pitágoras.




```
euclidean_distance = function(x1, x2){
    sqrt(sum((x1-x2)^2))
}
```

Autor: Ander Fernández

Programar el algoritmo kNN

1. Programar las medidas de distancia que queramos usar
2. Encontrar los k vecinos más cercanos
3. Hacer la predicción
4. Juntar todo

Programar el algoritmo kNN

Para encontrar los k vecinos más cercanos solo hay que:

```
euclidean_distance = function(x1, x2){
    sqrt(sum((x1-x2)^2))
}

nearest_neighbors = function(x,obs, k, FUN){









}
```

Autor: Ander Fernández

Programar el algoritmo kNN

Para encontrar los k vecinos más cercanos solo hay que:

- Calcular la distancia de nuestra observación respecto al resto.

```
euclidean_distance = function(x1, x2){
    sqrt(sum((x1-x2)^2))
}

nearest_neighbors = function(x,obs, k, FUN){

    # Calculamos las distancias
    dist = apply(x,1, FUN,obs)





}
```

Programar el algoritmo kNN

Para encontrar los k vecinos más cercanos solo hay que:

- Calcular la distancia de nuestra observación respecto al resto.
- Ordenar las observaciones según la distancia.

```
euclidean_distance = function(x1, x2){
    sqrt(sum((x1-x2)^2))
}

nearest_neighbors = function(x,obs, k, FUN){

    # Calculamos las distancias
    dist = apply(x,1, FUN,obs)

    # Encontramos los vecinos más cercanos
    distances = sort(dist)[1:k]



}
```

Programar el algoritmo kNN

Para encontrar los k vecinos más cercanos solo hay que:

- Calcular la distancia de nuestra observación respecto al resto.
- Ordenar las observaciones según la distancia.
- Quedarnos con los vecinos con la k distancia más corta.

```
euclidean_distance = function(x1, x2){
    sqrt(sum((x1-x2)^2))
}

nearest_neighbors = function(x,obs, k, FUN){

    # Calculamos las distancias
    dist = apply(x,1, FUN,obs)

    # Encontramos los vecinos más cercanos
    distances = sort(dist)[1:k]
    neighbor_ind = which(dist %in% sort(dist)[1:k])

    return(neighbor_ind)
}
```

Programar el algoritmo kNN

1. Programar las medidas de distancia que queramos usar
2. Encontrar los k vecinos más cercanos
3. Hacer la predicción
4. Juntar todo

Programar el algoritmo kNN

Datos Categóricos

En el caso de los datos categóricos, la predicción es el valor que más se repite.

Si el 9 de 10 vecinos no se han ido de la compañía... predigo que tú tampoco te irás de la compañía.

```
knn_prediction = function(x,y){

    x = as.matrix(x)

    # Calculamos la predicción en clasificación
    if(is.factor(x[,y]) | is.character(x[,y])){
       groups = table(x[,y])
       pred = names(groups[groups == max(groups)])
    }



}
```

Programar el algoritmo kNN

Datos Numéricos

En el caso de los datos numéricos la predicción es la media de los valores de nuestros vecinos.

Además, también se puede ponderar esa media con la distancia los vecinos, de tal forma que los vecinos más cercanos tengan más peso que los lejanos.

```
knn_prediction = function(x,y){

    x = as.matrix(x)

    # Calculamos la predicción en clasificación
    if(is.factor(x[,y]) | is.character(x[,y])){
       groups = table(x[,y])
       pred = names(groups[groups == max(groups)])
    }

    # Calculamos la predicción en regresión
    if(is.numeric(x[,y])){
       pred = mean(x[,y])
    }

    return(pred)
}
```

Programar el algoritmo kNN

1. Programar las medidas de distancia que queramos usar
2. Encontrar los k vecinos más cercanos
3. Hacer la predicción
4. Juntar todo

Programar el algoritmo kNN

Ahora simplemente debemos crear una función que haga todo este proceso de forma automática.

```
knn = function(x, obs, y, k, FUN){

  train_columns = which(colnames(x) != y)

   # Encontramos los vecinos más cercanos
   nn = nearest_neighbors(x = x[,train_columns],
                          obs = obs,
                          k = k,
                          FUN = FUN
                          )

    # Hacemos la predicción
    prediccion = knn_prediction(x[nn,], y = y)

    return(prediccion)

}
```

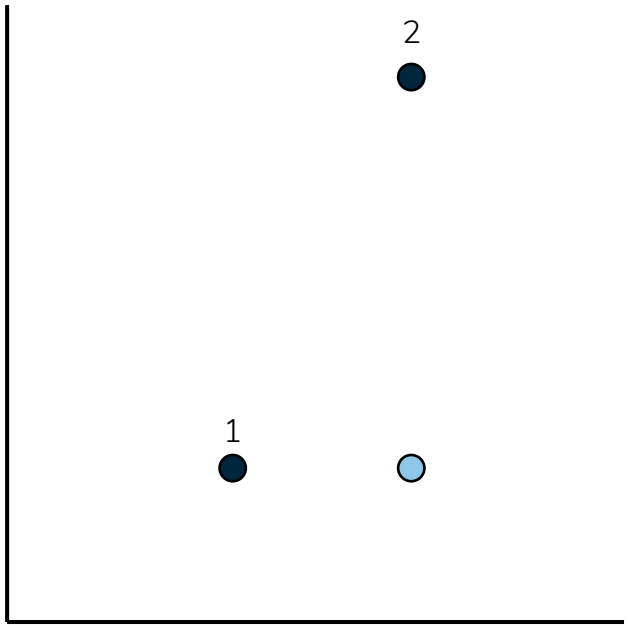
¡Veámos cómo funciona el algoritmo kNN en casos reales!

Cómo usar el algoritmo kNN

Escalas

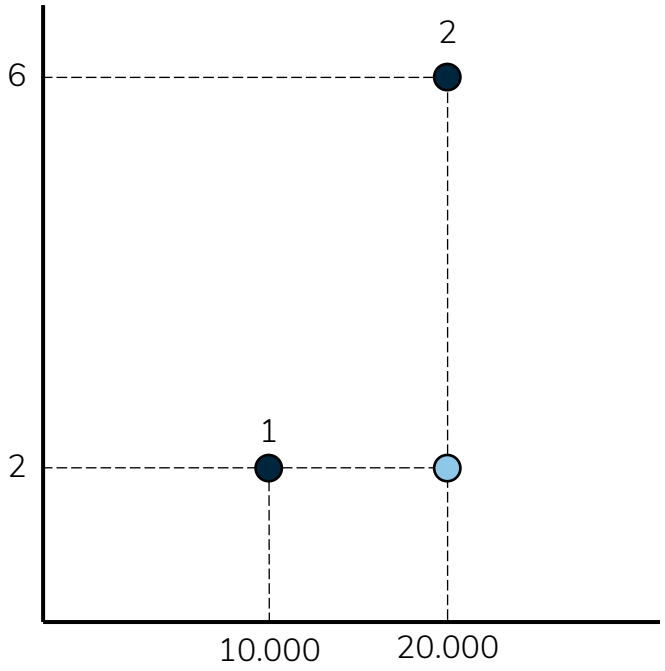
Autor: Ander Fernández

¿Cuál de los dos puntos oscuros está más cerca del claro?



Autor: Ander Fernández

Cuidado con las escalas...



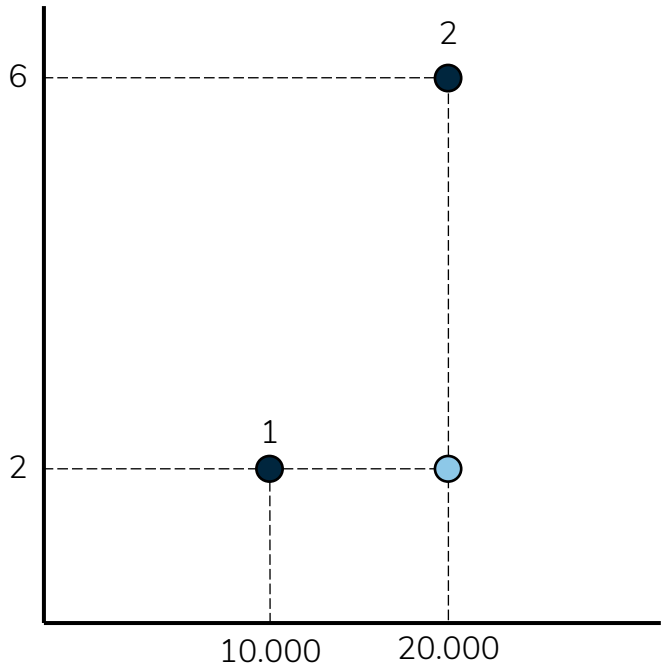
Autor: Ander Fernández

Cuidado con las escalas...

Distancia al punto 1: 20.000-10.000= 10.000

Distancia al punto 2: 6-2= 4

El punto 2 está más cerca del punto claro que el punto 1, debido a las escalas de los datos.

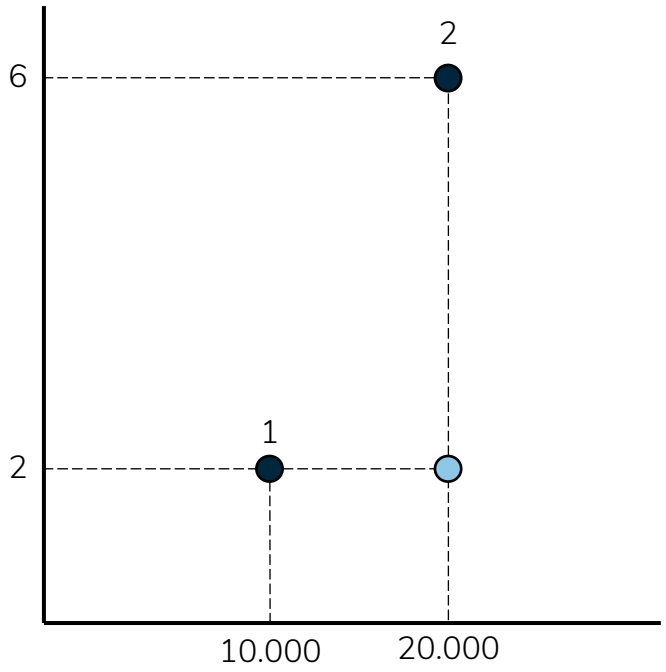


Cuidado con las escalas...

Distancia al punto 1: 20.000-10.000= 10.000

Distancia al punto 2: 6-2= 4

El punto 2 está más cerca del punto claro que el punto 1, debido a las escalas de los datos.



Moraleja

Si no todas las variables tienen la misma escala, el algoritmo no encontrará bien a los vecinos cercanos.

Por tanto, **para que el algoritmo kNN funcione bien, debemos asegurarnos de que todas las variables están en la misma escala**. A este proceso se le conoce como normalizar los datos.

Cómo normalizar datos

Normalizar los datos consiste en que los valores de una variable vayan de 0 a 1.

Para normalizar los datos simplemente se debe aplicar esta fórmula:

$$X_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
noralizar = function(x){
  (x - min(x))/(max(x) - min(x))
}
```

Autor: Ander Fernández

Cómo usar el algoritmo kNN

Elección de K

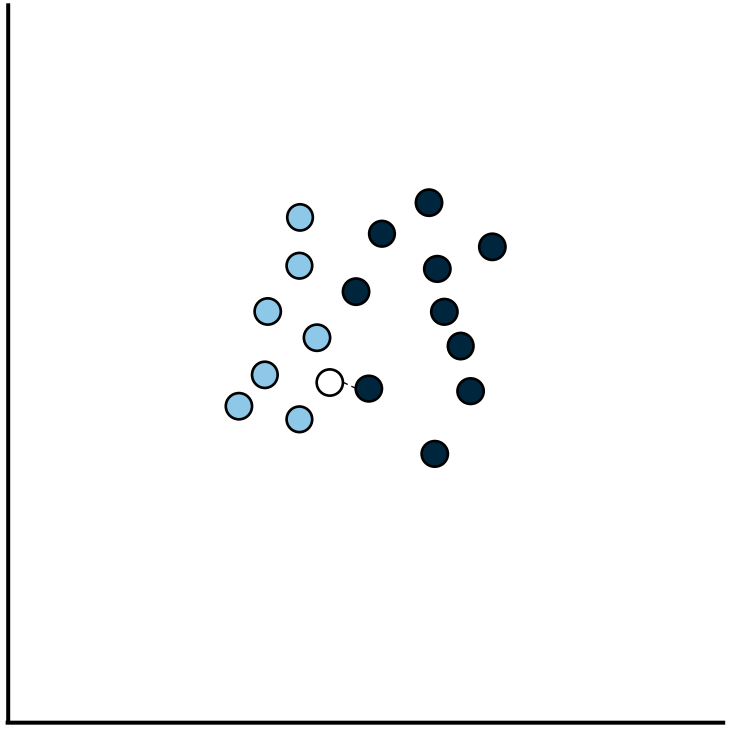
Autor: Ander Fernández

Elección de la K

Cuando k es baja (k = 1)

Si nos fijamos en un único vecino, puede que nuestro vecino no represente bien la realidad.

Con un valor de k baja, podemos sufrir de *overfitting*.



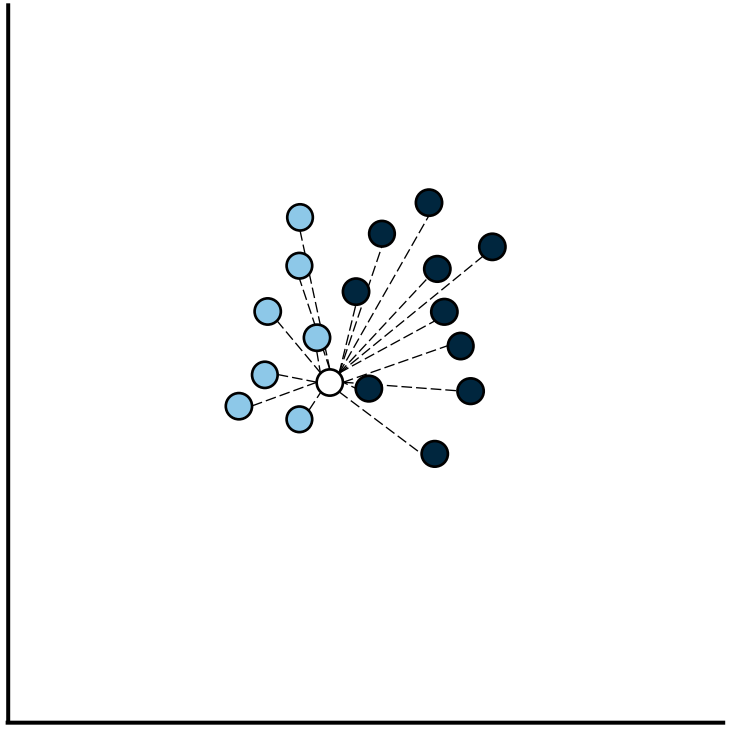
Elección de la K

Cuando k es alto (k = num observ)

Si nos fijamos en un todos los vecinos, siempre predeciremos lo mismo:

- En caso de clasificación, la moda, es decir, el valor más repetido.
- En caso de regresión, la media.

Con un valor de k alto, sufrimos de *underfitting*.



Elección de la K

Método 1

Como norma general, se puede elegir el número de *k* siguiendo la siguiente fórmula:

$$k = \sqrt{n}$$

Donde n es el número de observaciones.

Elección de la K

Método 1

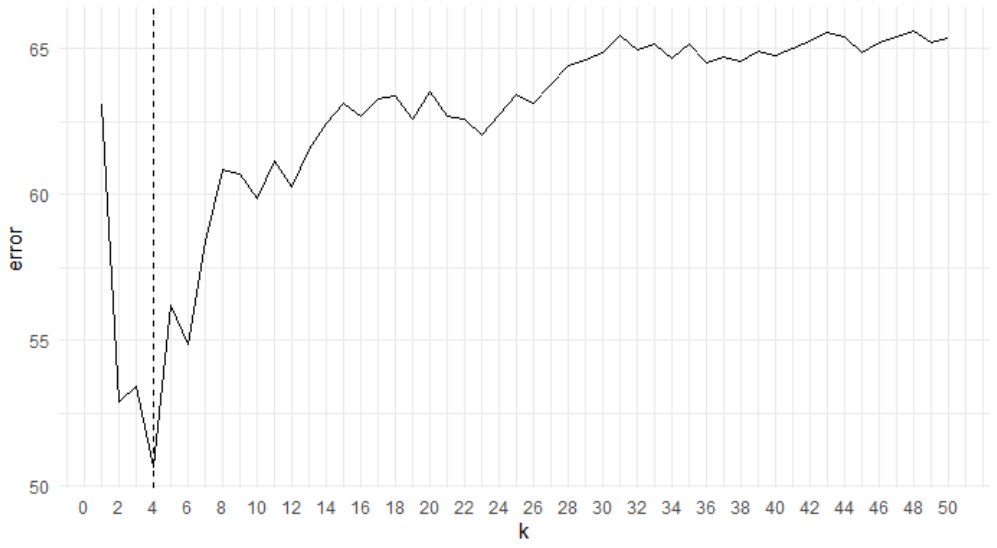
Como norma general, se puede elegir el número de *k* siguiendo la siguiente fórmula:

$$k = \sqrt{n}$$

Donde n es el número de observaciones.

Método 2

Obtener el error de la clasificación para diferentes valores de *k*, eligiendo la k que minimice el error.



Autor: Ander Fernández

Cómo usar el algoritmo kNN

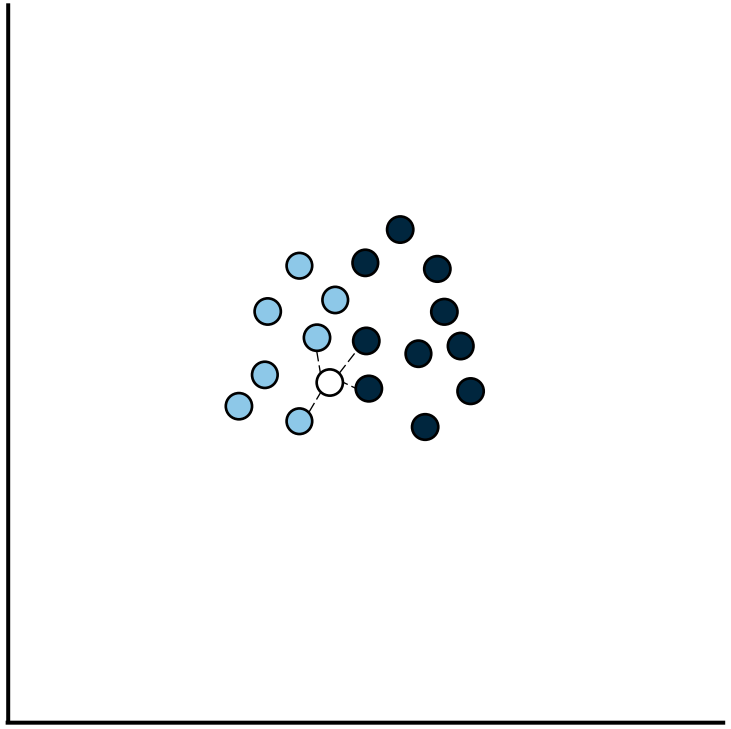
Cuidado con las votaciones...

Autor: Ander Fernández

Cuando se da un empate

Puede darse el caso de que a la hora de clasificar haya un empate en la votación.

En estos casos se incrementa la k en 1 hasta que se de un desempate.



Usar kNN en R

Paquete FNN

Autor: Ander Fernández

Usar kNN en R

Clasificación

Para usar kNN en problemas de clasificación, simplemente debemos pasar:

- Datos de train.
- Datos a predecir.
- Etiquetas de los datos de train.

```r
library(FNN)

# 1.Creamos los datasets
data.pred = iris[1,]
data.train = iris[2:nrow(iris), ]

# 2.Modificamos los datos
labels = data.train$Species
data.train$Species = NULL
data.pred$Species = NULL


FNN::knn(
  data.train,
  data.pred,
  cl = labels,
  k = 3
)
```

Usar kNN en R

Regresión

Para usar kNN en problemas de regresión, simplemente debemos pasar:

- Datos de train.
- Datos a predecir.
- Etiquetas de los datos de train.

```
library(FNN)
library(gapminder)

# 1.Creamos los datasets, igual que antes
library(gapminder)
columnas_elegir = c( "lifeExp","pop","gdpPercap")
datos_knn_num = gapminder[gapminder$year == "2007",
                          columnas_elegir]

data.pred = datos_knn_num[5,]
data.train = datos_knn_num[6:nrow(datos_knn_num), ]

# 2.Modificamos los datos
labels = data.train$lifeExp
data.train$lifeExp = NULL
data.pred$lifeExp = NULL

# 3. Hacemos la predicción
knn.reg(
  data.train,
  data.pred,
  y = labels,
  k = k
)
```

Casos de uso del algoritmo kNN

Casos de uso del algoritmo kNN

Imputación de valores perdidos

A veces los datasets tienen valores perdidos (NAs). Generalmente suele haber dos enfoques: eliminación de la variable o su imputación.

En el caso de la imputación, muchas veces se suele usar el algoritmo kNN, ya que funcionará mejor que la imputación de la media o moda.

Sistemas de Recomendación

Los sistemas de recomendación colaborativos se basan en encontrar personas similares, de tal forma que recomendamos contenido que a personas similares a nosotros les ha gustado y nosotros no hemos visto.

Para encontrar a esas personas similares se usa el algoritmo kNN.

¡Muchas Gracias!

github.com/anderfernandez/kNN-Deusto-Big-Data

Ander Fernández Jauregui

anderfernandez.com/blog